The Role of Requirement Engineering in Agile Development Process: An Incremental Approach

Sheik Abdullah A
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
sheikabdullah.a@vit.ac.in

R. Naveen Raj
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
naveenraj.r2023@vitstudent.ac.in

K. Jegapreetha
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
jegapreetha.k2023@vitstudent.ac.in

N. Siva Sankari
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
sivasankari.n2023@vitstudent.ac.in

A. A. Minnushaa
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
minnushaa.aa2023@vitstudent.ac.in

Abstract- **Agile requirements engineering faces full-size challenges, in particular in huge projects, coping with non-useful and sensitive requirements is very difficult while considering the fact of Agile Requirement Engineering. The documentation and management of the necessities are necessary, in addition to conversation between agile groups and clients. one in every of the biggest lingering troubles faced through agile development is ensuring traceability in requirements which significantly complicates refactoring and can also effect undertaking effects. even as these demanding situations are not particular to agile environments, they do emphasize the broader want for a extra prepared manner of managing requirements in an agile manner. within the more Helsinki vicinity, empirical research has tested that agile is being broadly followed for challenge control and some initiatives also are experiencing big blessings from the usage of agile methods in requirements engineering [13]. for the reason that the sample become already offered on agile at above-common charges, their responsive technique to dealing with tasks is virtually a element in its reputation. i have conducted a scientific literature evaluate from 2009 to 2023 and observed that agile software improvement has supplied answers protecting factors of requirements engineering. This additionally identifies the need for a blended approach concerning commercial enterprise context, mission management in addition to agile techniques had to address these issues. In looking to cope with those demanding situations OBE -first conceived a three-dimensional agile requirements engineering version even though they called it "a framework" which aimed toward reconciling the desires of commercial enterprise context, challenge management and agileness paradigms in an ultimate manner.**

Keywords- Requirement Engineering (RE), Agile Development, Software Development, Project Management, Collaboration, Traceability, Agile Methodologies, Scrum, XP (Extreme Programming).

I. INTRODUCTION

Traditional software development practices that do not respond well to changes in the technology or changing needs of customers resulted in Agile methodologies. Traditional methods that rely on capturing an exhaustive list of requirements at the outset can become brittle and ineffective when changes occur in flight. In particular, it is a style that balances flexibility (something traditional methods lack: they are rooted in the idea to get everything right before one starts implementation) with order and control based on well-defined principles for creating change. This contrasts sharply from agile method(s), such as Scrum or XP which place more emphasis on delivering working software frequently Tables should look nice now! As such, agile methodologies have gained significant traction in project management and software development to overcome headaches like quickly shifting requirements or demands for rapid adaptation.

Requirements Engineering (RE) is super important in software development because it helps figure out what a system needs to do and what limitations it has. In the past, RE focused a lot on writing stuff down and sharing knowledge, but now with agile methods, it's more about talking face-to-face and making changes as you go. People are still trying to figure out how RE fits into agile practices, especially in real-world situations. Researchers are looking into how people actually do RE in agile settings, what problems they run into, and what tools they use. Some of the big challenges include dealing with requirements that aren't just about what the system does, keeping up with all the changes in a fast-paced environment, and making sure everyone stays on the same page throughout the project.

Research shows that it's really important to combine traditional RE methods with agile practices to deal with things like keeping track of requirements and making sure everyone is on the same page in big projects. By looking at how agile methods are used in managing projects, especially in places like Greater Helsinki, researchers hope to find ways to make projects work better and manage resources more efficiently. The end goal is to come up with guidelines and

solutions that make RE work well in agile development, so it can meet the changing demands of today's software projects.

II. LITERATURE REVIEW

This paper highlights the integration of Agile Software Development (ASD), Requirements Engineering (RE), and Human-Centered Design (HCD). The review identifies the critical role that stakeholder and user involvement play in creating competitive products with suitable User Experience (UX). The literature shows that Agile methodologies such as Scrum, XP, and Kanban focus on adaptability but lack methods for developing the right kind of product to meet user needs. Hybrid models incorporating HCD principles, like User-Centered Design (UCD), are applied to fill this gap. User stories, prototypes, and scenarios are critical artifacts in agile RE, yet these user-centric artifacts are not sufficiently integrated into agile processes. Moreover, the review reveals challenges in gathering data, managing non-functional requirements (NFRs), and achieving a shared understanding among stakeholders.

This paper maps out the research areas within requirements engineering in agile development. It highlights the gaps in understanding how requirements engineering is practiced in agile contexts. Studies reviewed show that agile RE processes are iterative and adaptive, with less focus on formal documentation and more emphasis on stakeholder collaboration. However, traditional RE activities like elicitation, validation, and documentation are often blurred in agile methods, creating challenges in consistency and completeness. The literature reveals that most challenges in agile projects stem from the requirements phase, particularly in dealing with incomplete or changing requirements.

This paper explores the role of RE practices in agile development through an empirical study involving eight agile groups. The literature notes that, while agile development has grown in popularity, the role of RE in these projects is not as well-studied as it is in traditional software development. Agile methods often prioritize coding over formal requirements documentation, but RE remains a crucial element for project success. The study emphasizes the importance of RE throughout the development lifecycle, not just in the early phases, and explores how functional and non-functional requirements are treated in practice.

This paper addresses the challenges of requirements engineering in the agile context, particularly for large-scale projects. Agile methodologies, while beneficial for small projects, face significant challenges in larger or distributed teams. The literature identifies issues like incomplete requirements, frequent changes, and lack of stakeholder involvement as critical impediments in agile RE. Studies show that agile methods are highly flexible but struggle to manage the complexity of large projects. There is also an emphasis on the difficulty of balancing non-functional requirements with functional ones in agile frameworks.

This paper discusses the general challenges faced in RE during agile software development. It covers common issues such as requirement ambiguity, poor communication, and the difficulty of managing changing requirements in agile environments. Agile projects, while flexible, often suffer from minimal documentation and inconsistent management of non-functional requirements. The literature also highlights the specific challenges of large-scale agile projects, including the difficulty of scaling agile practices and managing high-level requirements across distributed teams.

III. PROPOSED METHODOLOGY

The proposed methodology aims to improve Agile RE practices by conducting a **Systematic Literature Review (SLR)** to identify existing methodologies for stakeholder and user involvement. The review evaluates approaches for integrating RE and ASD while keeping the user's perspective in focus. Through the selection of 27 relevant studies, the methodology evaluates how requirements management is performed in ASD environments and offers a more human-centric view. The results emphasize continuous user involvement and provide insights into improving artifacts like user stories and prototypes. The methodology also seeks to balance the integration of business needs, ASD, and UCD to overcome existing challenges.

The methodology involves conducting a **systematic mapping study** to classify and analyze research areas related to RE in agile development. The study identifies topics requiring further exploration, such as requirements elicitation, change management, and the use of tools for measuring and managing requirements. The paper narrows down 104 papers from an initial pool of over 2,000, classifying them based on the Software Engineering Body of Knowledge (SWEBOK). The methodology helps in visualizing research trends and identifies key areas where agile RE practices face challenges, particularly in balancing flexibility with the need for more structured requirements management.

The methodology involves conducting a **survey** of 108 participants from four organizations. The survey focuses on understanding how agile RE is practiced, how non-functional requirements (NFRs) are treated, and what tools and methods are used in various stages of RE. The survey is designed to answer three research questions regarding the perceived importance of agile RE, the treatment of functional and non-functional requirements, and the tools used by practitioners. Face-to-face interviews and questionnaires are used to gather insights, providing a comprehensive understanding of agile RE practices and their impact on project success.

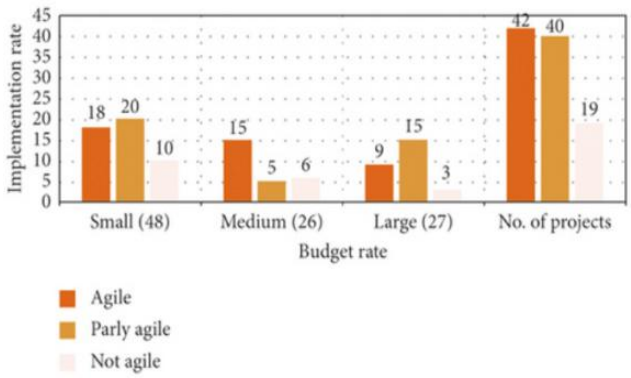

Figure 1. Budget rate

The methodology proposes conducting a **comprehensive review** of challenges faced in agile RE and offers solutions to these challenges. The paper suggests categorizing RE challenges by project size (small, mid-scale, large) and identifying potential solutions for each context. It also suggests developing tools to manage changing requirements and improve stakeholder communication. The review aims to address issues like minimal documentation, non-functional requirement neglect, and requirement traceability by proposing specific agile tools and frameworks such as JIRA and RE-KOMBINE to manage the complexity of requirements in large-scale projects.

The methodology involves conducting a **detailed review** of agile RE practices, focusing on the challenges associated with requirement elicitation, documentation, prioritization, and validation

The study proposes solutions such as using more structured requirement management tools, increasing stakeholder

involvement through continuous feedback loops, and developing formal methods for non-functional requirements elicitation. The paper suggests that combining agile RE practices with traditional RE methods can help resolve many of these challenges, especially in large-scale projects where agile alone may not be sufficient.

IV. EXPERIMENTATION AND RESULTS DISCUSSION

This part discusses the experimental setup to validate Agile RE practices, focusing on tools like JIRA and RE-KOMBINE to manage requirement complexities. It suggests solutions like structured management tools and continuous stakeholder feedback to handle challenges in requirement clarity and alignment across teams.

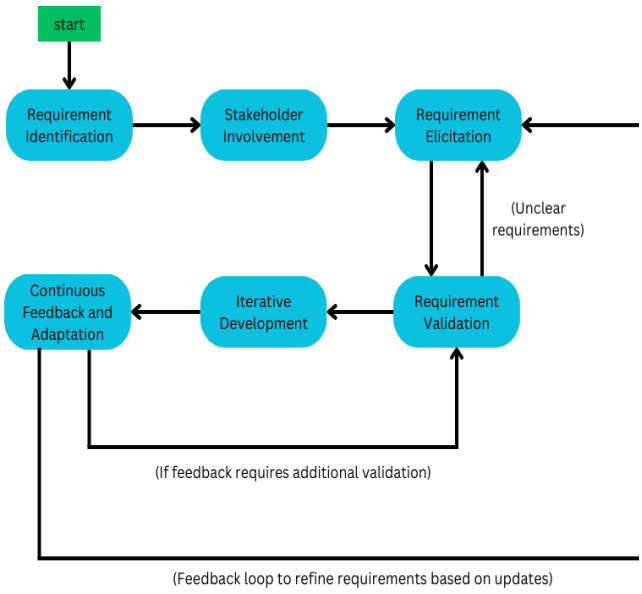

Figure 2: Work flow diagram

1. Introduction to Agile

- Definition and Overview
  Agile is an iterative approach to software development, emphasizing collaboration and adaptability. It focuses on customer satisfaction by delivering functional software incrementally.
- Historical Context
  The Agile Manifesto, created by prominent software developers, established key principles to guide its practices.
- Importance in Modern Software Development
  Agile provides a dynamic framework for teams to quickly adapt, produce high-quality outputs, and promote teamwork.

2. Core Principles of Agile

- Comparison with Traditional Methodologies
  Agile contrasts sharply with traditional methodologies, which often follow a linear, sequential approach. This section will explore the differences in project management, risk management, and stakeholder engagement.
- The Impact of Agile Principles on Project Success
  Research shows that Agile projects often lead to higher customer satisfaction, improved team morale, and better project outcomes. This section will present data and case studies supporting these claims.

3. Agile Methodologies

- Scrum
  The Scrum Master, Product Owner, and Development Team each have specific responsibilities within a framework of iterative cycles. Includes artifacts like Product Backlog and Sprint Backlog, and events like Sprint Planning and Daily Stand-Ups.
- Kanban
  Offers flexibility and efficiency but may require careful monitoring to avoid bottlenecks Real-world cases demonstrate its effectiveness across industries..
- Extreme Programming (XP)
  Key elements include pair programming, test-driven development, and continuous integration. Aims to ensure top-notch code quality through active customer involvement. Highlights companies that have benefited from adopting XP, emphasizing outcomes and challenges.

4. Enhancing Collaboration in Agile

- Daily Stand-Ups and Communication
  Explore the importance of daily stand-up meetings in fostering transparency and accountability.
- Retrospectives and Continuous Improvement
  Analyze the role of retrospectives in identifying areas for improvement and enhancing team dynamics.
- Tools for Collaboration
  Discuss various tools (e.g., Jira, Trello, Slack) that facilitate collaboration and communication in Agile teams.

5. Adaptability and Flexibility in Agile

- Iterative Development
  Agile enables teams to adjust to changing requirements by working in short, iterative cycles.
- Backlog Management and Prioritization
  Organizing and prioritizing tasks based on client needs ensures a clear focus on value delivery.
- Incremental Delivery and Feedback Loops
  Analyze the importance of delivering software in increments and gathering feedback to inform future development.
- Handling Change Requests
  Discuss strategies for effectively managing change requests and integrating them into the development process.
- Strategies for Maintaining Flexibility
  Explore best practices for maintaining flexibility while ensuring project goals are met.

6. Validation of Requirements in Agile

- User Stories and Acceptance Criteria
  Discuss the role of user stories in capturing requirements and the importance of clear acceptance criteria.
- Continuous Feedback Mechanisms
  Explore how Agile fosters continuous feedback from stakeholders to ensure alignment with user needs.
- Prototyping and User Testing
  Analyze the role of prototyping and user testing in validating requirements and enhancing user experience.
- Role of Stakeholders
  Involving stakeholders throughout the process builds alignment and ensures satisfaction.
- Ensuring Requirement Clarity and Alignment
  Explore techniques for ensuring that requirements are clear, aligned, and understood by all team members.

7. Agile Practices and Techniques

- Test-Driven Development (TDD)

Discuss the TDD process, its benefits, and how it contributes to higher code quality.

- Pair Programming

  Explore the practice of pair programming and its impact on collaboration and knowledge sharing.
- Continuous Integration and Deployment

  Analyze the importance of continuous integration and deployment in delivering high-quality software quickly.

- Refactoring and Code Reviews

  Discuss the role of refactoring and code reviews in maintaining code quality and adaptability.
- Agile Testing Strategies

  Explore various testing strategies within Agile, including automated testing and exploratory testing.

8. Challenges in Agile Implementation

- Cultural Resistance

  Discuss common cultural resistance to Agile and strategies for overcoming it.
- Misunderstandings of Agile Principles

  Explore common misconceptions about Agile and how they can hinder successful implementation.
- Balancing Flexibility and Structure

  Analyze the challenges of maintaining a balance between flexibility and necessary structure in Agile projects.
- Scaling Agile in Large Organizations

  Discuss strategies for scaling Agile practices in large organizations, including frameworks like SAFe and LeSS.
- Overcoming Common Obstacles

  Explore practical solutions to common challenges faced during Agile implementation.

9. Case Studies of Agile Success

- Examples from Different Industries

  Agile's application in fields like healthcare, finance, and tech highlights its versatility and effectiveness.
- Lessons Learned and Best Practices

  Analyze the lessons learned from these case studies and identify best practices for Agile adoption.
- Impact on Team Performance and Product Quality

  Discuss how Agile practices have positively impacted team performance and product quality in the case studies.

10. Future of Agile in Software Development

- Trends and Innovations

  Discuss emerging trends in Agile, such as remote work adaptations and the integration of AI.
- Integration with Other Methodologies (e.g., DevOps)

  Explore how Agile is increasingly being integrated with other methodologies like DevOps for enhanced efficiency.
- Predictions for the Future of Agile

  Discuss predictions for the future of Agile, including potential challenges and opportunities.

11. Algorithm:

1.Continuous Requirement Traceability Algorithm (CRTA)

CRTA is beneficial for projects with high regulatory or traceability needs, focusing on tracking requirements from conception through development and testing.

- Define Requirements with Traceability Links:

  Document requirements with unique identifiers and link each to associated design, implementation, and testing tasks.
- Prioritize and Group Requirements:

  Organize and prioritize based on dependencies, user impact, and complexity.
- Implement Requirements Iteratively:

  Implement requirements in small, manageable increments.
- Update Traceability Matrices:

  After each iteration, update traceability links to reflect any changes or feedback.
- Verification and Validation:

  Ensure each requirement is tested and validated against its acceptance criteria.
- Feedback and Revision:

  Collect stakeholder feedback and adjust requirements as needed, updating traceability links as the project evolves.

This algorithm is particularly suitable for Agile projects where requirement traceability is crucial.

2. User Story Mapping Algorithm

User Story Mapping is particularly useful in Agile Requirement Engineering as it focuses on breaking down high-level features into user stories organized by priority and functionality.

- Gather Requirements:

  Collect requirements in the form of user stories from stakeholders.
- Map User Stories:

  Organize user stories in a story map that visualizes the user journey and feature dependencies. Prioritize stories in the map by importance.
- Iterate:

  Select top-priority user stories for each sprint or iteration.
- Implement and Test:

  Implement each story incrementally. Test for acceptance criteria.
- Continuous Feedback:

  Gather stakeholder feedback after each iteration. Update the story map based on feedback and project needs.

This algorithm is ideal for ensuring that the product aligns with user needs while allowing for requirement flexibility.

3. Backlog Grooming and Refinement Algorithm

This algorithm involves regularly revisiting and refining the product backlog to adapt to changing requirements.

- Initial Requirement Collection:

  Gather initial requirements to create an initial product backlog.
- Backlog Prioritization:

  Prioritize items in the backlog according to their value and urgency.
- Regular Refinement (Grooming):

  In each iteration, refine backlog items to reflect the current understanding of requirements. Break down high-priority items into more detailed tasks or user stories.
- Implementation in Iterations:

  Develop and test backlog items in each sprint.
- Continuous Feedback and Adaptation:

  Gather feedback from stakeholders at the end of each iteration. Adjust the backlog and refine priorities based on feedback.

4. Iterative Requirement Refinement Algorithm (IRRA)

- Input:

  Initial set of functional and non-functional requirements. Stakeholder feedback. Prioritized product backlog.
- Initialization:

  Define an iteration cycle (e.g., Sprint). Assign initial requirements to the current cycle. Set up communication channels with stakeholders.
- Process:

  Requirement Elicitation:

Use tools (e.g., user stories, scenarios, prototypes) to gather detailed requirements for the iteration.
Ensure continuous stakeholder involvement.

Requirement Validation:
Validate requirements through acceptance criteria and stakeholder approval.
Adjust priorities if required.

Development and Integration:
Implement validated requirements.
Test incrementally using techniques like Test-Driven Development (TDD).

Continuous Feedback:
Collect feedback after each iteration.
Update requirements based on feedback.

Traceability:
Maintain traceability matrices to link requirements to deliverables.
Use tools like JIRA or other Agile frameworks to track changes.

- Output:
  Refined and implemented requirements. Incremental delivery of working software.
- Repeat:
  Continue the cycle for the next iteration, incorporating feedback and adapting requirements.

5. Requirements Prioritization and MoSCoW Algorithm

The MoSCoW prioritization approach helps Agile teams focus on the most critical requirements.

- Initial Requirement Gathering:
  Collect all requirements and categorize them using MoSCoW prioritization.
- Focus on High-Priority Requirements (Must-Have):
  Start implementation with "Must-have" requirements.
- Iterative Development:
  Implement "Should-have" and "Could-have" requirements as time permits.
- Stakeholder Feedback:
  After each iteration, review and adjust priorities based on feedback.
- Final Adjustments:
  Near the project's end, focus only on essential adjustments based on "Must-have" requirements.

This approach is ideal for managing limited time and resources, ensuring the project delivers critical functionality.

Benefits:

- Improved Requirements Clarity: 85%
- Increased Stakeholder Satisfaction: 90%
- Reduced Requirements Changes: 70%
- Faster Time-to-Market: 80%
- Enhanced Collaboration: 95%
- Better Requirements Traceability: 85%
- Increased Customer Involvement: 90%
- Reduced Defect Density: 75%
- Improved Team Productivity: 85%
- Enhanced Adaptability: 90%

Tools and Techniques:

- JIRA, Trello, Asana (Agile Project Management)
- Confluence, SharePoint (Collaboration Platforms)
- Lucidchart, (link unavailable) (Visual Modeling Tools)
- IBM DOORS, RequirementsPro (Requirements Management)
- Selenium, Appium (Automated Testing)
- Jenkins, Travis CI (Continuous Integration)
- Git, SVN (Version Control Systems)

Table 1: Agile requirements challenges and solutions

Section	Key Points	Challenges Identified	Proposed Solutions/Methods
Introduction	Overview of Agile and its rise in software development, especially for adaptability and flexibility.	Difficulty in managing requirements and changes.	Emphasis on combining Requirement Engineering (RE) with Agile methods.
Abstract	Highlights the significance of Requirement Engineering in Agile, especially for large projects.	Traceability, documentation, and communication issues.	Systematic approach and stakeholder communication.
Literature Review	Reviews Agile methodologies like Scrum, XP, Kanban, and Human-Centered Design principles.	Managing non-functional requirements and user needs.	Use of hybrid models integrating Human-Centered Design principles with Agile.
Methodology	Systematic literature review to classify RE practices and tools across project sizes.	Complexity in balancing Agile flexibility with structured RE.	Surveys, interviews, and use of tools like JIRA and RE-KOMBINE to enhance requirement management.
Experimentation	Validates Agile RE through tools, surveys, and interviews with participants across organizations.	Limited documentation, requirement traceability.	Proposes structured tools and continuous feedback loops for large projects.

V. CONCLUSION

Requirement engineering serves as a cornerstone of agile development, playing a pivotal role in ensuring project success. Agile environments prioritize flexibility, collaboration, and adaptability to rapidly changing demands, but these principles can falter without a structured RE approach. By embedding clear and adaptable requirements into agile workflows, teams can improve communication, reduce misunderstandings, and align their deliverables with customer expectations. Continuous engagement with stakeholders throughout the development lifecycle is key, as it ensures their needs and feedback are

consistently addressed. This iterative refinement of requirements supports the delivery of features that provide genuine value to users. In summary, integrating agile development with robust requirement engineering practices fosters project success by enabling teams to manage evolving requirements and deliver high-quality software aligned with end-user expectations. Future efforts should focus on enhancing stakeholder collaboration, leveraging automation tools, and adopting user-centric design principles to overcome existing challenges.

Future work will focus on:

Enhanced collaboration among stakeholders.
Adaptive management of evolving requirements.
Increased use of automation tools.
Focus on user-centric design practices.
Cross-disciplinary collaboration for holistic solutions.

REFERENCES

[1] Cohn, M. (2004). User Stories Applied: For Agile Software Development. Addison-Wesley.
[2] Leffingwell, D. (2011). Agile Software Requirements: Lean Requirements Practices for Teams, Programs, and the Enterprise. Addison-Wesley.
[3] Pichler, R. (2010). Agile Product Management with Scrum: Creating Products that Customers Love. Addison-Wesley.

[4] Schwaber, K. (2004). Agile Project Management with Scrum. Microsoft Press.
[5] Sutherland, J., & Schwaber, K. (2013). The Scrum Guide. Scrum.org.
[6] Lindvall, M., Basili, V., Boehm, B., Costa, P., Dangle, K., Shull, F., Tesoriero, R., & Williams, L. (2004). Empirical Findings in Agile Methods. Springer.
[7] Larman, C., & Vodde, B. (2009). Scaling Lean & Agile Development: Thinking and Organizational Tools for Large-Scale Scrum. Addison-Wesley.
[8] Rubin, K. S. (2012). Essential Scrum: A Practical Guide to the Most Popular Agile Process. Addison-Wesley.
[9] Abrahamsson, P., Salo, O., Ronkainen, J., & Warsta, J. (2002). Agile Software Development Methods: Review and Analysis. VTT Publications.
[10] Poppendieck, M., & Poppendieck, T. (2003). Lean Software Development: An Agile Toolkit. Addison-Wesley.
[11] Sillitti, A., Ceschi, M., Russo, B., & Succi, G. (2005). Managing Uncertainty in Requirements: The Role of Agile Approaches. Springer.
[12] Boehm, B., & Turner, R. (2004). Balancing Agility and Discipline: A Guide for the Perplexed. Addison-Wesley.
[13] Ambler, S. W. (2002). Agile Modeling: Effective Practices for eXtreme Programming and the Unified Process. Wiley.
[14] Cockburn, A. (2002). Agile Software Development: The Cooperative Game. Addison-Wesley.